Machine Learning

Harry Morrison

May 2025

1 Fundamentals

1.1 Definitions

Machine Learning	ML algorithms are tools for the automatic acquisition of knowledge. Machine Learning is the science (and art) of programming computers so they can learn from data. Is also known as inductive learning.
Inductive Learning	A form of logical inference that allows you to obtain generic conclusions about a particular set of examples.
Classification	Predicting a nominal set of labels L_1, L_2.
Regression	Predicting a real value.

1.2 Learning Types

Supervised Model that identifies the relationship between input features and the target variable within a given dataset. We have a target function $f : X \to y$ that is $y = f(x)$ and we have labelled input examples (x_1, y_1). The model selects $g \approx f$ from the hypothesis space.

- Linear Regression
- Logistic Regression
- K-Nearest Neighbours
- Support Vector Machines (SVMs)
- Decision Trees and Random Forests

Unsupervised Model that identifies relationships in the data without labelled data.

- Clustering
  - Hierarchical Cluster Analysis (HCA)
  - k-means clustering
  - Density-based spatial clustering of applications with noise (DBSCAN)
- Visualisation and dimensionality reduction
  - Principal Component Analysis (PCA)
  - Locally-Linear Embedding (LLE)
  - t-distributed Stochastic Neighbour Embedding (t-SNE)
- Association rule learning
  - Apriori algorithm
  - Eclat algorithm

Semi-Supervised Models that learn off partially labelled data (few labelled instances) because labelling is time consuming and costly.

Self-Supervised Models that generate a set of labels from unlabelled data.

Reinforcement The learning system, called an agent, can observe the environment, select and perform actions, and get rewards in return (or penalties in the form of negative rewards). It must then learn by itself what is the best strategy, called a policy, to get the most reward over time. A policy defines what action the agent should choose when it is in a given situation.

1.3 Generalisation Types

Instance Based Learning Memorises the training examples, and generalise to new cases by using a similarity measure to compare to the learned examples.

Model Based Learning Build a model from the training examples, and use that model to make predictions.

1.4 Challenges of Machine Learning

Insufficient quantity of training data ML algorithms require lots of data.

Non-representative or poor-quality data Data needs to be able to represent the cases you want to generalise:

- Sampling Noise: Unrepresentative by chance with a small sample set.
- Sampling Bias: When sampling method is flawed.
- Errors, Outliers, Noise: Remove features, pad or fill blanks, compare models that use subsets of the features.

Irrelevant features Garbage in, Garbage out. Feature engineering can fix that:

1. Feature Selection: choosing features
2. Feature Extraction: combining features

Overfitting Happens when the model is too complex relative to the amount and noisiness of the training data. Here are possible solutions:

- Simplify the model by selecting one with fewer parameters, reducing the number of attributes in the training data, or constraining the model.
- Gather more training data.
- Reduce the noise in the training data (e.g., fix data errors and remove outliers).

Underfitting Occurs when your model is too simple to learn the underlying structure of the data. For example, a linear model of life satisfaction is prone to underfit; reality is just more complex than the model, so its predictions are bound to be inaccurate, even on the training examples. Here are the main options for fixing this problem:

- Select a more powerful model, with more parameters.
- Feed better features to the learning algorithm (feature engineering).
- Reduce the constraints on the model (for example by reducing the regularisation hyperparameter).

1.5 Splitting the Data

- Models need to generalise to new cases.
- Train your model using the training set, and you test it using the test set.
- The error rate on the test set is called the generalisation error or out-of sample error.

1.6 Multiclass Classification

- **Binary**: Discriminating betweeen two classes.
- **Multiclass**: direct methods (Softmax, Random Forest, Naive Bayes) or reductions (one-vs-all, one-vs-one).
- **Multilabel**: instances can have multiple binary labels (e.g. multi-topic tagging, yes/no).
- **Multi-output Multiclass**: multiple labels per instance, each with more than two classes (e.g. image denoising pixel prediction, a car type and colour).

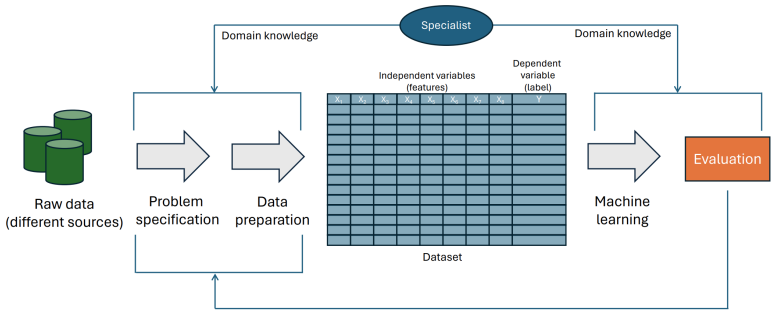
Task	Output	Cardinality per output
Binary classification	1	2
Multiclass classification	1	> 2
Multilabel classification	> 1	2
Multioutput-multiclass classification	> 1	> 2

Table 2: Comparison of classification task types

2 Machine Learning Projects

2.1 End to End Machine Learning Project

1. Understand the problem and check assumptions
2. Visualise and explore the data
3. Prepare the data for a ML algorithm
4. Present your solution
5. Launch, monitor, and keep checking assumptions



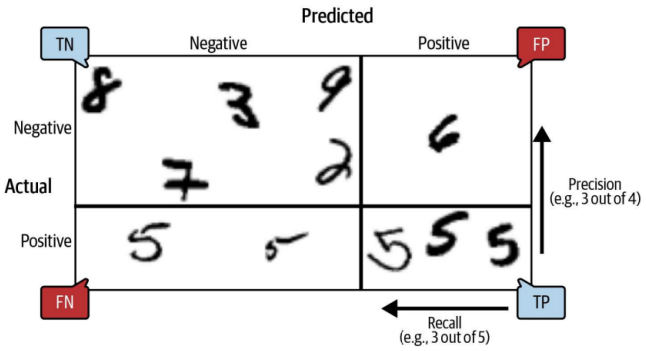
3 Performance Metrics

3.1 Distance Based

Mean Squared Error	Mean Absolute Error
l_2 norm $\|\|x\|\|_2$	l_1 norm $\|\|x\|\|_1$
$MSE(X, f) = \frac{1}{m}\sum_{i=1}^{m}(f(x_i) - y_i)^2$	$MAE(X, f) = \frac{1}{m}\sum_{i=1}^{m}\|f(x_i) - y_i\|$
Higher weight to large differences	Equal weight to differences
Differentiable	Non-differentiable
Quadratic	Linear
Squared units	Same units

3.2 Confusion Matrix

To evaluate a Classifier, count the number of instances class A is classified as class B.



Metric	Formula	Interpretation
Precision	$P = \frac{TP}{TP+FP}$	Of all predicted positives, how many are correct?
Recall	$R = \frac{TP}{TP+FN}$	Of all actual positives, how many are found?
F1 score	$F_1 = \frac{2PR}{P+R}$	Harmonic mean of precision and recall
Accuracy	$\text{Acc} = \frac{TP+TN}{\text{Total}}$	Out of all predictions, how many were correct?
Specificity	$\text{TNR} = \frac{TN}{TN+FP}$	True negative rate, how many negative instances were negative?
FPR	$\text{FPR} = \frac{FP}{FP+TN} = 1 - TNR$	Negative instances that are incorrectly classified as positives

3.2.1 Precision-Recall Curve

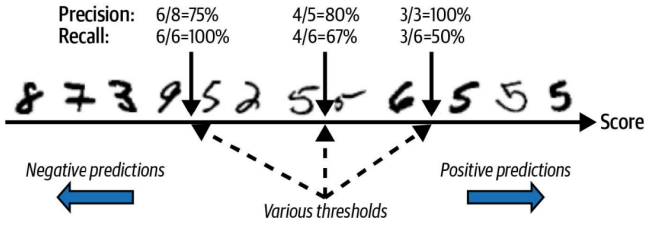
In some scenarios, false positives can be costly, so precision is more important.

- Predicting that it is safe to change lanes while driving, when it is not.

In some scenarios, false negatives can be costly, so recall is more important.

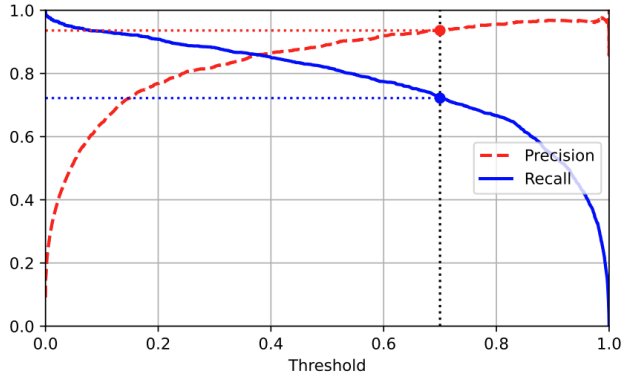
- Predicting that a patient does not have cancer when they do.

Adjusting the threshold for accept and reject tunes what trade-off is desired

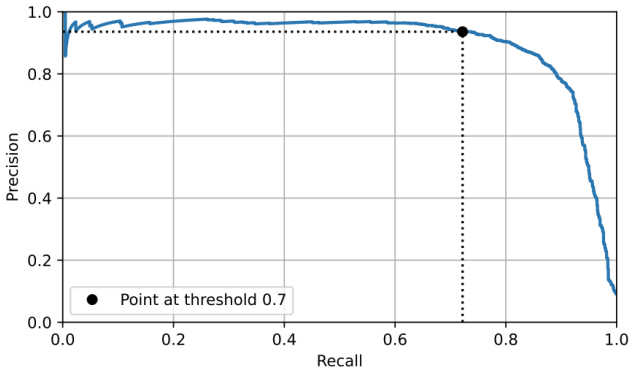


3.3 Receiver Operating Characteristic (ROC) Curve

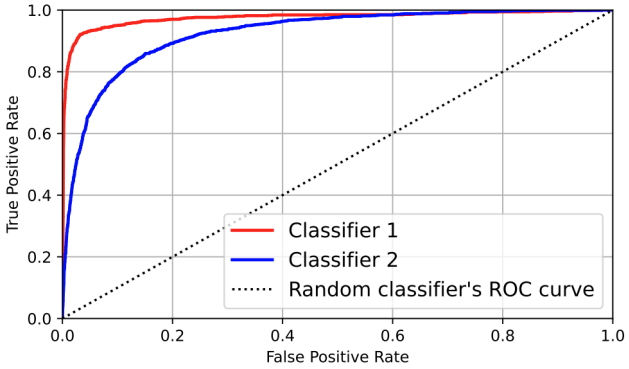
Plots the true positive rate (recall) against the false positive rate (FPR) for varying threshold settings. A good classifier will have a large **area under the curve (AUC)**. A bad classifier will be a linear line (Random classifier).



(a) Precision-Recall Curve



(b) Precision VS Recall Curve



3.4 ROC vs PR curve

Use the PR curve when the positive class is rare or when avoiding false positives matters most; otherwise, use the ROC curve. ROC curves can look deceptively strong in imbalanced datasets because the abundance of negatives inflates performance, whereas the PR curve reveals any real shortfall by showing how close the classifier is to the ideal top-right corner.

4 Regression Models

4.1 Linear Regression

Making a prediction by computing a weighted sum of the input features, plus a constant called the bias term (also called the intercept term).

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + ... + \theta_n x_n$$
$$\hat{y} = h_\theta(\mathbf{x}) = \theta^T \mathbf{x}$$

Training the model can be done by a closed form solution, **The Normal Equation**, or by **Gradient Descent**.

4.1.1 Normal Equation

Training means setting the parameters so a model best fits the training set. We will then need to find the value of θ that minimises the costs function $\mathrm{C}(\theta)$:

$$\hat{\theta} = \arg_\theta \min \mathrm{C}(\theta)$$

Then we need to pick a performance measure (or cost function) to minimise. This is usually done with Residual Sum of Squares (RSS), which is the total squared error.

$$RSS(X, h_\theta) = \sum_{i=1}^{m} (\theta^T x_i - y_i)^2$$

From there we need to find the value of θ that minimises the RSS.

$$\begin{aligned} RSS(\mathbf{X}, h_\theta) &= ||\mathbf{y} - \mathbf{X}\theta||^2 \\ &= (\mathbf{y} - \mathbf{X}\theta)^T(\mathbf{y} - \mathbf{X}\theta) \end{aligned}$$

Differentiate with respect to θ, and set derivative to zero to find the minimum and, finally, solve for *theta*. This gives us **The Normal Equation**:

$$\begin{aligned} \frac{\partial \text{RSS}}{\partial \theta} &= -2\,\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\theta) \\ 0 &= \mathbf{X}^\top(\mathbf{y} - \mathbf{X}\theta) \\ \hat{\theta} &= (\mathbf{X}^\top\mathbf{X})^{-1}\,\mathbf{X}^\top\mathbf{y} \end{aligned}$$

Beware: running time of normal equation can be cubic in number of features but linear with respect to the number of instances in the training set. Predictions are fast.

4.2 Gradient Descent

General idea is to tweak parameters iteratively in order to minimise a cost function

- Measures local gradient of the error function with regards to the parameter vector θ and it goes in the direction of the descending gradient.
- Starts by filling θ with random values.

4.2.1 Implementation

Calculate how much the cost function will change is you change θ_j just a little bit. This is called the *Partial Derivative*.

$$\frac{\partial}{\partial \theta_j}\text{MSE}(\theta) = \frac{2}{m}\sum_{i=1}^{m}\left(\theta^\top x^{(i)} - y^{(i)}\right)x_j^{(i)}$$

The idea:

- When $\frac{\partial}{\partial \text{z}} f(\text{z})$ is negative at $\text{z}^{(t)}$, then z^{t+1} need to be $\text{z}^{(t)}$ + something to go to the minimum of $f(\text{z})$.
- When $\frac{\partial}{\partial \text{z}} f(\text{z})$ is positive at $\text{z}^{(t)}$, then z^{t+1} need to be $\text{z}^{(t)}$ − something to go to the minimum of $f(\text{z})$.

The gradient vector contains all the partial derivatives of the cost function: $\text{grad}(\theta) = \nabla_\theta MSE(\theta) = \begin{pmatrix} \frac{\partial}{\partial \theta_0}\text{MSE}(\theta) \\ \frac{\partial}{\partial \theta_1}\text{MSE}(\theta) \\ \vdots \\ \frac{\partial}{\partial \theta_n}\text{MSE}(\theta) \end{pmatrix} = \frac{2}{m}\,\mathbf{X}^\top(\mathbf{X}\theta - \mathbf{y})$ Finally, with a learning rate, η, the gradient descent step is:

$$\theta^{(t+1)} \;=\; \theta^{(t)} \;-\; \eta\,\nabla\!\left(\theta^{(t)}\right)$$

4.2.2 Limitations

- Can fall into local minimum
- Can get stuck or slow on plateaus
- The MSE cost function is sensitive to scaling
- Learning rate can be too small and may never reach the minimum
- Learning rate can be too big and overstep the minimum

4.2.3 GD Methods

Batch GD Uses the whole training set to compute the gradients at every step, which makes it very slow when the training set is large.

Stochastic GD Just picks a random instance in the training set at every step and computes the gradients based only on that single instance. Makes the algorithm much faster since it has very little data to manipulate at every iteration. Also possible to train on huge training sets.

Mini-batch GD At each step, computes the gradients on small random sets of instances (mini-batches). Get a performance boost from hardware optimization of matrix operations (e.g., vectorisation, GPUs).

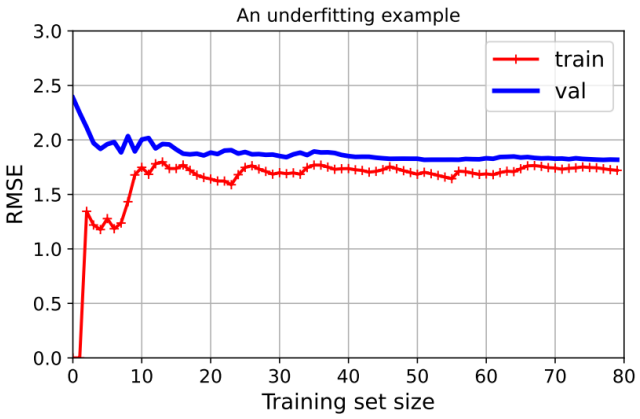
Algorithm	Large m	Large n	Hyperparams	Scaling	Scikit-Learn
Normal equation	Fast	Slow	0	No	N/A
SVD	Fast	Slow	0	No	`LinearRegression`
Batch GD	Slow	Fast	2	Yes	N/A
Stochastic GD	Fast	Fast	≥ 2	Yes	`SGDRegressor`
Mini-batch GD	Fast	Fast	≥ 2	Yes	N/A

Table 4: Comparison of different linear-regression solution algorithms

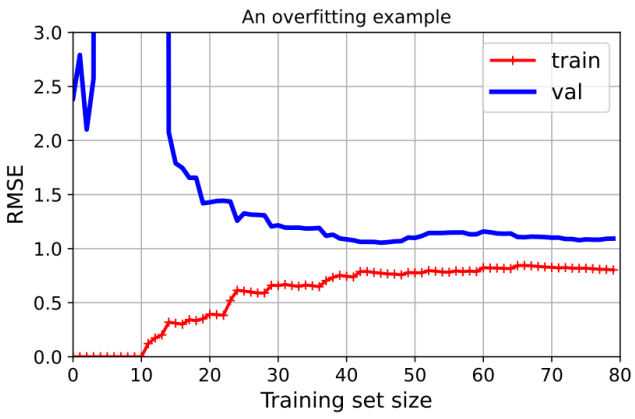
4.3 Learning Curves

Learning curves are plots of the model's performance which has training error and validation error as a function of the training set size.

- Underfitting can be seen if the error rates for both training and validation is high, close together, and a straight line plateau.
- Overfitting can be seen if the error rates are lower and there is a gap between the curves (curves may converge with more training examples).



(a) Underfitting Learning Curve



(b) Overfitting Learning Curve

4.4 Logistic Regression

- Classification example of regression. Estimates the probability that an instance belongs to a particular class.
- Can solve by **closed form** or **gradient descent** solutions.
- Predicts the probability:

$$\hat{p} = h_\theta(\mathbf{x}) = \sigma(\theta^T \mathbf{x}) = \frac{1}{1 + e^{-\theta^T \mathbf{x}}}$$

- Then classification is made by a threshold, 1 if $\theta^T \mathbf{x}$ is positive, and 0 if negative:

$$\hat{y} = \begin{cases} 0, & \text{if } \hat{p} < 0.5, \\ 1, & \text{if } \hat{p} \geq 0.5. \end{cases}$$

4.4.1 Cost Function

We want to find the parameter vector θ so that high probabilities are for positive classes and low probabilities for negative classes. Cost function for a single instance:

$$C(\theta) = \begin{cases} -\ln(\hat{p}), & \text{if } y = 1, \\ -\ln(1 - \hat{p}), & \text{if } y = 0. \end{cases}$$

The **Logistic Regression cost function (log loss)** is:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \Big[y^{(i)} \log\big(\hat{p}^{(i)}\big) + \big(1 - y^{(i)}\big) \log\big(1 - \hat{p}^{(i)}\big) \Big].$$

- There is no known closed form solution for this equation
- This cost function is convex, so GD is guaranteed to find the global minimum (if learning rate not large and there are enough iterations)

5 Bias, Variance, and Regularisation

5.1 Bias Variance Trade off

- **Generalisation error** = Bias + Variance + Irreducible error
- **Bias**: Error from wrong model assumptions (e.g. assuming linear when data is quadratic)
  - High bias → underfitting problem
  - High bias algorithms tend to be less flexible with strong assumptions about the target function → lower predictive performance.
- **Variance**: model sensitivity to small training set changes
  - High variance → overfitting problem
  - High variance algorithms tend to be more flexible with weaker assumptions about the target function → may account for every single training example
- **Tradeoff**:

$$\begin{cases} \uparrow \text{Complexity} \ \Rightarrow \downarrow \text{Bias}, \uparrow \text{Variance} \\ \downarrow \text{Complexity} \ \Rightarrow \uparrow \text{Bias}, \downarrow \text{Variance} \end{cases}$$

- **Remedies**:
  - **Underfitting**: increase model complexity, engineer better features, reduce regularisation
  - **Overfitting**: gather more data, use cross-validation, perform feature selection or reduce dimensions, increase regularisation

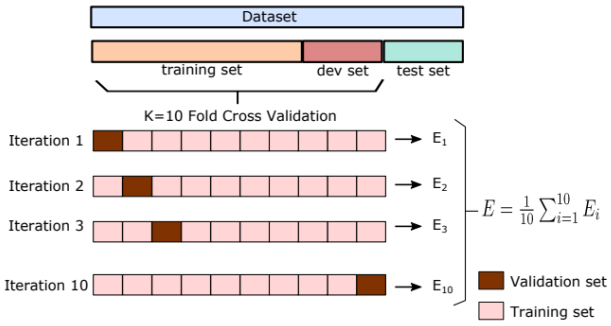
5.2 Fine Tuning

- A hyperparameter is a parameter of a learning algorithm (not in the model) and must be set before training.
- Using the test set to pick the best hyperparameter values tends to make the model not perform well on other new data than the test set.
- We need a validation set

5.2.1 K-Fold Cross Validation

Goal: Split training set into k folds, rotate which fold serves as validation, average the results.

- Error rate on the test set is an estimation of the generalisation error
- **Grid Search:** An exhaustive automated search over all hyperparameter combinations `GridSearchCV`.
- **Random Search:** A randomised search of hyperparameter combinations when search space is large `RandomizedSearchCV`.
- Finally, retrain the model with the best parameters on the entire training set, then evaluate on test set.



5.2.2 Early Stopping

- For iterative learners (e.g. SGD), monitor validation error and stop when it ceases to improve.
- Example: use `SGDRegressor.partial_fit(...)` in a loop, snapshot the model with the lowest validation RMSE.
- *Note*: with stochastic and mini-batch gradient descent, the curves are not so smooth.

5.3 Regularised Linear Models

Goal: Constrain weight magnitudes to reduce overfitting. Hyperparameters:

- a controls the strength of the penalty.
- r controls the weight of lasso regression in elastic net.

5.3.1 Ridge Regression ℓ_2

- Shrinks weights smoothly toward zero.
- Added to cost function in training only.
- Cost function:

$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_i \theta_i^2$$

5.3.2 Lasso Regression ℓ_1

- Can drive some weights exactly to zero (feature selection).
- Cost function:

$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_i |\theta_i|$$

5.3.3 Elastic Net

- Combines Ridge and Lasso regression, shrinking weights and feature selection.
- Cost function:

$$J(\theta) = \text{MSE}(\theta) + r\,\alpha \sum_i |\theta_i| + \tfrac{1-r}{2}\,\alpha \sum_i \theta_i^2$$

5.3.4 Softmax Regression

- Logistic regression can be generalised to support multiple classes directly.
- This is called Softmax or Multinomial Logistic Regression
- **Scores**: $s_k(x) = \theta_k^\top x$ for each class $k = 1, \ldots, K$.
- **Softmax**:

$$\hat{p}_k = \frac{e^{s_k(x)}}{\sum_{j=1}^{K} e^{s_j(x)}}.$$

- **Prediction**: $\hat{y} = \arg\max_k \hat{p}_k = \arg\max_k s_k(x)$.
- **Cost**:

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^{m} \sum_{k=1}^{K} y_k^{(i)} \log \hat{p}_k^{(i)}.$$

- Decision boundaries between class i and j are when $s_i(x) = s_j(x)$.

6 K-Nearest Neighbours (K-NN) Algorithm

- Instance based algorithm → no explicit training and stores all examples in memory.
- **Distance metrics**:

$$D(x_i, x_j) = \Big(\sum_{\ell=1}^{n} |x_{i\ell} - x_{j\ell}|^p \Big)^{1/p},$$

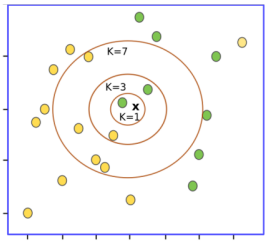
with $p = 1$ (Manhattan), $p = 2$ (Euclidean).

We can decrease the importance of an individual instance ($k > 1$).

Majority vote in the set of k nearest neighbours.

$$\hat{y}_q = \arg\max_{c \in C} \sum_{i=1}^{k} \delta(c_i, c),$$

where $\delta(a, b) = 1$ if $a == b$ and $\delta(a, b) = 0$ if $a \neq b$.



- **Prediction**:
  - **1-NN**: label of the nearest neighbor.
  - k-**NN**: majority vote among the k nearest.
  - **Weighted** k-**NN**: weight votes by $1/d^2$.
- **Pros & Cons**:
  - ✓ Simple and intuitive
  - × Memory and compute intensive at prediction time.
  - × Sensitive to feature scales ⇒ always scale/normalise.

  - $\times$ Sensitive to outliers and noise
  - Choice of k trades off bias vs. variance.
    * Small $k \rightarrow$ risk of overfitting and high variance
    * Large $k \rightarrow$ risk of underfitting and high bias
- **Variations**:
  - Approximate Nearest Neighbour (k-d trees)
  - Locally sensitive Hashing (LSH) - for high dimensions
  - Density based k-NN

7 Support Vector Machines (SVM)

- **Definition**: A Support Vector Machine is a versatile model that can produce both linear and nonlinear decision boundaries.
- **Applications**:
  - Binary classification
  - Regression
- **Strengths**:
  - Effective on complex, moderate-sized datasets
  - Finds a "largest-margin" boundary, which often generalises well

7.1 Linear SVM Classification

1. **Large-Margin Intuition**
   - Seeks two parallel hyperplanes ("the street") that separate the classes, maximising the distance (margin) between them.
   - Only the points lying on the edges of that margin ("support vectors") determine the boundary.
   - Adding extra points far from the margin does not change the decision boundary.
2. **Support Vectors**
   - Training instances exactly on the margin edges.
   - They "support" or fix the position of the separating hyperplane.
3. **Sensitivity to Feature Scales**
   - SVMs use inner products to measure distances; unscaled features can distort margins.
   - Always fit a scaler (e.g. `StandardScaler`) on the training set, then apply the same transform to validation/test.
   - After scaling, the margin boundary is more stable and meaningful.

7.2 Hard vs. Soft Margin Classification

1. **Hard-Margin SVM**
   - Strictly requires all instances to lie on or outside the margin, with no misclassification.

- Works only if the data are perfectly linearly separable.
- Extremely sensitive to outliers—one mislabeled (or extreme) point can make a solution impossible.

2. **Soft-Margin SVM**
   - Introduces slack variables $\zeta^{(i)} \geq 0$ to allow some margin violations.
   - **Objective**: minimise
   $$\frac{1}{2}\|w\|^2 \; + \; C\sum_{i=1}^{m}\zeta^{(i)},$$
   subject to
   $$t^{(i)}\big(w^T x^{(i)} + b\big) \; \geq \; 1 - \zeta^{(i)}, \quad \zeta^{(i)} \geq 0.$$
   - **Hyperparameter** C:
     - Large $C \rightarrow$ narrow margin, fewer violations (more emphasis on correct classification)
     - Small $C \rightarrow$ wider margin, more violations allowed (more emphasis on large margin)
3. **Visualising the Trade-off**
   - Low C (e.g. 1) $\rightarrow$ wider street, allows some misclassifications (more regularisation)
   - High C (e.g. 100) $\rightarrow$ tight street, fewer violations (less regularisation)

7.3 Nonlinear Decision Boundaries

1. **Polynomial Features Approach**
   - Explicitly add polynomial combinations (e.g. x_1^2, $x_1 x_2$, x_2^2, etc.) until the transformed dataset becomes (approximately) linearly separable.
   - Example: for a two-moon dataset, adding degree-3 polynomial features can produce a linear separation in the expanded feature space.
2. **Polynomial Kernel Trick**
   - Instead of explicitly constructing polynomial features, define
   $$K(a, b) = (\gamma\, a^T b + r)^d,$$
   which directly computes the dot product in the transformed space.
   - Example: `SVC(kernel="poly", degree=3, coef0=1, C=5)`
   - Much more efficient since high-dimensional expansions remain implicit.
3. **Similarity (Gaussian RBF) Features**
   - Choose a set of "landmark" points $\{\ell_j\}$.
   - For each training instance x, compute features
   $$\phi_\gamma(x, \ell_j) = \exp\big(-\gamma\|x - \ell_j\|^2\big).$$
   - Intuition: measure "closeness" to each landmark. If data are not linearly separable in the original space, they may become separable in this similarity space.
4. **Gaussian RBF Kernel Trick**
   - Instead of explicitly computing similarity features, define
   $$K(a, b) = \exp\big(-\gamma\|a - b\|^2\big).$$

- Example: `SVC(kernel="rbf", gamma=5, C=0.001)`
- Adjustments to γ and C dramatically change decision boundary shapes and regularisation strength.

7.4 SVM for Regression (SVR)

1. **Principle**
   - Instead of separating classes, SVR fits a "tube" (analogous to a margin) of width 2ϵ, trying to keep as many points as possible within that tube while penalising those outside.
   - Uses similar hinge-style loss:

$$\text{loss}(y, \hat{y}) = \max\big(0,\ |y - \hat{y}| - \epsilon\big).$$

2. **Linear SVR**
   - Example code:

```
from sklearn.svm import LinearSVR
svm_reg = LinearSVR(epsilon=1.5)
svm_reg.fit(X, y)
```

   - Controls how many points can lie outside the $\pm\epsilon$ tube via the regularisation parameter C.
3. **Nonlinear SVR**
   - Use a kernelised SVR (e.g. polynomial, RBF) to capture nonlinear relationships.
   - Tuning C, γ (for RBF), and ϵ controls smoothness vs. fit.

7.5 7. "Under the Hood" (High-Level Math)

1. **Decision Function (Linear Case)**
   - Let $\mathbf{w} \in \mathbb{R}^n$ be the weight vector, and b the bias. For a new point x, the decision score is:

$$s(x) = \mathbf{w}^T x + b.$$

   - Prediction rule (binary):

$$\hat{y} = \begin{cases} +1, & \text{if } s(x) \geq 0, \\ -1, & \text{if } s(x) < 0. \end{cases}$$

2. **Primal Optimization (Hard-Margin)**
   - Minimise $\frac{1}{2}\|\mathbf{w}\|^2$ subject to $t^{(i)}\big(\mathbf{w}^T x^{(i)} + b\big) \geq 1$ for all i.
   - Ensures that all points lie outside or on the margin with no violations.
3. **Primal Optimization (Soft-Margin)**
   - Introduces slack variables $\zeta^{(i)} \geq 0$ for each training point.
   - Minimise $\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \zeta^{(i)}$ subject to

$$t^{(i)}\big(\mathbf{w}^T x^{(i)} + b\big) \geq 1 - \zeta^{(i)}, \quad \zeta^{(i)} \geq 0.$$

4. **Hinge Loss Interpretation**

- The soft-margin objective is equivalent to minimising

$$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m}\max\big(0,\ 1 - t^{(i)}(\mathbf{w}^T x^{(i)} + b)\big).$$

5. **Dual Formulation**

- Solve for dual variables $\alpha^{(i)} \geq 0$ by minimising

$$\frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha^{(i)}\alpha^{(j)}t^{(i)}t^{(j)}\langle x^{(i)},\, x^{(j)}\rangle \ -\ \sum_{i=1}^{m}\alpha^{(i)},$$

  subject to $\sum_i \alpha^{(i)}t^{(i)} = 0$.

- Dual is advantageous when $m < n$, and it directly enables the kernel trick.

6. **Recovering $\mathbf{w}, b$ from α**

- Once optimal $\widehat{\alpha}$ are found, compute

$$\widehat{\mathbf{w}} = \sum_{i=1}^{m}\widehat{\alpha}^{(i)}\, t^{(i)}\, x^{(i)}.$$

- Compute $\widehat{b}$ by averaging over support vectors (those with $\widehat{\alpha}^{(i)} > 0$):

$$\widehat{b} = \frac{1}{n_s}\sum_{\{i:\widehat{\alpha}^{(i)}>0\}}\big(t^{(i)} - \widehat{\mathbf{w}}^T x^{(i)}\big).$$

7.6 The Kernel Trick

1. **Motivation**

- Rather than explicitly mapping data via $\phi(x)$ into a higher-dimensional space, define a kernel function

$$K(a,b) = \langle\phi(a),\, \phi(b)\rangle$$

  which computes the inner product in feature space directly from the original vectors a, b.

2. **Common Kernels**

- **Linear kernel**:

$$K(a,b) = a^T b.$$

- **Polynomial kernel (degree d)**:

$$K(a,b) = (\gamma\, a^T b + r)^d,$$

  where γ scales the dot product, r is the "coef0" term.

- **Gaussian RBF kernel**:

$$K(a,b) = \exp\big(-\gamma\|a-b\|^2\big).$$

- **Sigmoid (hyperbolic tangent) kernel**:

$$K(a,b) = \tanh\big(\gamma\, a^T b + r\big).$$

3. **Kernelised Decision Function**

- For a test point $x^{(n)}$,

$$f\big(x^{(n)}\big) = \sum_{i=1}^{m}\widehat{\alpha}^{(i)}\, t^{(i)}\, K\big(x^{(i)},\, x^{(n)}\big) \ +\ \widehat{b}.$$

- Only support vectors ($\widehat{\alpha}^{(i)} > 0$) contribute to the sum.

7.7 Alternative Training via Gradient Descent

- Instead of solving the QP in dual form, one can minimise the hinge-loss primal objective with (stochastic) gradient descent:

$$J(\mathbf{w}, b) = \frac{1}{2}\|\mathbf{w}\|^2 \; + \; C \sum_{i=1}^{m} \max\big(0,\; 1 - t^{(i)}(\mathbf{w}^T x^{(i)} + b)\big).$$

- Implemented in sklearn as `SGDClassifier(loss="hinge")`.
- Converges more slowly than dedicated QP solvers, but scales well to very large datasets.

7.8 Practical Tips & Takeaways

- **Always scale features** before training any SVM.
- **Tune C and γ (for RBF)** via cross-validation.
  - Low $\gamma \to$ smoother decision boundary (points far apart still "similar").
  - High $\gamma \to$ very wiggly boundary that can overfit.
- **Choose linear SVM (LinearSVC)** when you suspect data are (approximately) linearly separable in raw or scaled space, especially for large m.
- **Use SVC with kernel** only when you need a nonlinear boundary and dataset size remains moderate.
- **Inspect support vectors** if you want to understand which points are critical to the boundary.
- **For regression**, SVR often yields robust fits, but tuning ϵ and C is crucial to control bias–variance trade-off.

8 Decision Trees and CART

8.1 Decision Trees

- **Idea**:
  - Versatile and highly interpretable
  - Classification and Regression Trees (CARTs) split data using predictor variables where end nodes have the prediction for the target value
  - Series of questions
  - Overall idea is to find out which feature is more informative to ask questions about, and consider the effects of different answers to these questions
- **Terminology**:
  - ***Split (or internal) node*** - within the tree and directs instances. The split label (of the form $X_j \leq t_j$) indicates the left-hand branch resulting from that split.
  - ***Branch*** - links connecting nodes
  - ***Leaf (or terminal) node*** - Node at end of branch where prediction is done.
- **Making Predictions**:
  - Tree nodes also tell us the number of training samples in each node by class.

- ***Gini Impurity***:

$$G_i = 1 - \sum_{k=1}^{n} p_{i,k}^2$$

  - Where $p_{i,k}^2$ is the ratio of class k instances among the training instance in the i^{th} node.
  - low if most of the training instances on that node are from just one class

8.2 CART Training Algorithm

- Classification And Regression Tree (CART) algorithm is used to train decision trees.
- Divide the feature space - the set of possible values for $X_1, X_2, ..., X_n$ - into J distinct and non-overlapping regions, $R_1, R_2, ..., R_J$
- Consider the regions as boxes which are determined by recursive binary splitting
- Top down (starts at top of tree/root node)
- ***Greedy Algorithm***: searches for optimal split at the node level, then repeats the process recursively at subsequent levels, but overall solution is not guaranteed to be optimal.
- The algo finds the best predictor X_j and cut point t_j that minimises the cost function:

$$R_1(j, t_j) = \{X | X_j \leq t_j\} \text{ and } R_2(j, t_j) = \{X | X_j \geq t_j\}$$

- Repeat process for new subspaces and stop when it cannot find a split that will reduce impurity or when stopping condition is satisfied.
- **Classification** - The class with the highest probability given the training instances in R_J
  - CART searches for the pair (X_j, t_j) that produces the purest subsets.
  - Cost function: $J(X_j, t_j) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right}$
- **Regression** - Mean of the responsible values for the training instances in R_j
  - DTs are more suitable for non-linear and complex relationships
  - Cost function: $J(X_j, t_j) = \frac{m_{left}}{m} MSE_{left} + \frac{m_{right}}{m} MSE_{right}$
  - Where: $MSE_{node} = \frac{\sum_{i \in node} (\hat{y}_{node} - y_i)^2}{m_{node}}$
  - And: $\hat{y}_{node} = \frac{\sum_{i \in node} y_i}{m_{node}}$
- **Prone to Overfit** - without pruning or regularisation.
- **Sensitive to rotation** - diagonal vs straight.
- **High variance models** - Small changes to hyperparameters or to data may produce very different models.
- **Entropy**

$$H_i = -\sum_{k=1}^{n} p_{i,k} \log_2(p_{i,k})$$

  - A sets entropy is 0 when it contains instances of only one class
  - Most of the time produces similar results to Gini impurity
  - Gini impurity is slightly faster to compute
  - Entropy tends to produce slightly more balanced trees

8.3 Regularisation and Pruning

- DTs make few assumptions about the training data: they are thus referred to as nonparametric models, very (maybe too) adaptive.
- As a results, tree structure is likely to overfit.
- A smaller tree with fewer splits (regions) might lead to lower *variance* and better interpretation at the cost of *bias*.
- Need to restrict maximum depth or prune.
- **Regularisation Parameters:**
  - `max_depth`
  - `min_samples_split`
  - `min_samples_leaf`
  - `min_weight_fraction_leaf`
  - `max_leaf_nodes`
  - `max_features`
  - Increasing `min_*` or reducing `max_*`, regularises DT.
- **Pruning**
  - We want to select the tree with the lowest error on the test set. We could: use cross-validation on every possible sub-tree, or grow a large tree and prune it back.
  - Consider a sequence of trees according to a tuning hyperparameter α. For each value of α there corresponds a subtree $\mathrm{T} \subset \mathrm{T}_0$ that minimises:

$$\sum_{l=1}^{|\mathrm{T}|} \sum_{x_i \in \mathrm{R}_l} (y_i - \hat{y}^2_{\mathrm{R}_l}) + \alpha|\mathrm{T}|$$

  - Where, $|\mathrm{T}|$ is the number of terminal nodes of the subtree T, R_l is the rectangle corresponding to the lth terminal node, and $\hat{y}_{\mathrm{R}_l}$ is the predicted response of R_l.
  - α controls a trade-off between the subtree's complexity and its fit to the training data.
    - $\alpha = 0$: T will equal T_0 (just measuring training error)
    - Increasing α: reduces terminal nodes favouring smaller subtree $\rightarrow$ reduce overfit.
  - **Steps**:
    1. Build a regression tree on the training data.
    2. Obtain a sequence of subtrees with different number of terminal nodes by varying α.
    3. Perform Cross-validation to estimate the average validation error for each α or subtree.
    4. Choose α that minimises the average error.

9 Ensemble Learning

- Combine predictions of a group of predictors to obtain a single and potentially better model than by using the best individual model.
- Wisdom of the crowd

- **Resampling Methods**:
  - Involves repeatedly drawing samples from a training set and fitting a model of interest on each sample to obtain additional information about the fitted model. Two of the most common resampling methods are:
    1. Cross validation
    2. Boostrap
  - We want a model that produces a small test error and has low variability (similar consistent results).

9.1 Voting Classifiers

- Aggregating the predictions of classifiers to predict the class.
- Even if each classifier is a weak learning, the ensemble can be a strong learner provided there are enough weak learners with sufficient diversity.
- Diversity can be achieved through using different algorithms
- **Hard Voting**: Ensemble's predicted class is the majority voted class.
- **Soft Voting**: Ensemble's predicted class is the class that receives the highest average class probability. However, predicting probabilities can be expensive.

9.2 Bagging and Pasting

The idea is to use the same training algorithm for every predictor but train them on different random subsets of the training set.

Bagging Sampling with replacement

Pasting Sampling without replacement

Random Patches Features and instances are sampled.

Random Subspaces Features are sampled.

Out of Bag Evaluation

- Each boostrap, some observations will be omitted from the training set.
- On average, only $\frac{2}{3}$ of training instances are sampled for each predictor.
- The remaining $\frac{1}{3}$ are known as out-of-bag instances and are not the same for all predictors.
- We can use the OOB instances to form our validation set to use as much of our training data as possible.
- OOB Evaluation Steps:
  1. For each training instance x_i, find estimators where x_i is OOB instance and use those predictors to predict label of x_i.
  2. Aggregate those predictions (classification or regression)
  3. Repeat for all training instances.
  4. Compare all OOB predictions to true labels.
     - ***OOB classification error***
     - ***OOB MSE regression***

9.3 Random Forests

A Random Forest is an ensemble of Decision Trees, generally trained by the bagging method. `RandomForestClassi`

- Recall that in DT the very best feature is selected and searched when splitting a node.
- In RFs, extra randomness is introduced when splitting a node
- Greater tree diversity, by trading a higher bias for a lower variance.
- Searches for best feature among a random subset of features.
- Samples $\sqrt{n}$ features (where n is total number of features) to get subset.

Extremely Randomized Trees:

- Trades more bias for lower variance
- Uses random thresholds for each feature when splitting a node.
- faster to train than regular RF.

Feature Importance Feature importance can be used to provide overall summary about the contribution of each feature to the trees.

- For a given feature, the relative feature importance is:
  - Average total decrease in MSE (for regression trees)
  - Average total decrease in Gini index (for classification trees)
- Weighted sum of the nodes that split from the feature node.

9.4 Boosting

9.5 Stacking

10 Dimensionality Reduction

- **Curse of Dimensionality**: Training sometimes has a large number of features which means its takes and long time to train and is difficult to find a good solution.
- **Goal**: Reduce the number of dimensions by learning the latent, underlying structure of the data.
- Unsupervised learning technique that can be used as a preprocessing step.
- **Drawbacks**:
  - System performs slightly worse
  - Makes system more complex and harder to maintain
- **Benefits**:
  - Reduces number of dimensions
  - Can reduce noise and avoid overfitting
  - Useful for visualisation in high dimensional data

10.1 Main Approaches

- **Projection**: Find a low-dimensional flat (a d-dimensional hyperplane) that captures the most salient structure of the data.
- **Manifold Learning**: Assume the data lie on a curved surface "manifold") embedded in $\mathbb{R}^n$; try to unfold that surface into $\mathbb{R}^d$ while preserving chosen geometric relationships.

10.2 Principal Component Analysis

Why PCA?:

- *Linear* dimensionality-reduction
- Finds the hyperplane projection matrix capturing the **maximum variance** of centred data $\mathbf{X}_c$.
- Reduces computational cost and eases visualisation, but can reduce performance.
- Can compress datasets.

Concepts

- **Principal axis (i-th)**: the i-th orthogonal direction of greatest remaining variance.
- **Principal component (PC)**: the coordinates of a data point after projection onto those axes.
- **Sign ambiguity**: an axis has no inherent direction; both $\pm\mathbf{v}_i$ are valid.

Computing the Axes

$$\mathbf{X}_c = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top, \qquad \mathbf{W}_d = \begin{bmatrix}\mathbf{v}_1 & \dots & \mathbf{v}_d\end{bmatrix}$$

where the columns of $\mathbf{V}$ (right-singular vectors) are the principal axes and the top d of them are retained.

Projecting the Data

- **Reconstruction Error**: The mean squared distance between the original data and the reconstructed data. Applied through inverse transformation.

$$\mathbf{Z} = \mathbf{X}_c\,\mathbf{W}_d$$
$$\mathbf{X}_{\text{recon}} = \mathbf{Z}\,\mathbf{W}_d^\top + \boldsymbol{\mu}$$

Choosing the Dimensionality d

- **Variance threshold:** keep PCs until cumulative variance $\geq 95\%$ (`PCA(n_components=0.95)`).
- **Elbow plot:** inspect the cumsum of explained variance for a knee where extra PCs add little.

Feature Normalisation

- Rrecommended to scale the features to have unit standard deviation before apply PCA.

Variants for Large or Complex Data

Variant	When to use	Benefit / Complexity
Randomised PCA	Huge feature sets, need only first d PCs	$O(md^2) + O(d^3)$; drastic speed-ups for $d \ll n$
Incremental PCA	Data too large for RAM / streaming batches	Calls `partial_fit` on chunks; constant memory
Kernel PCA	Non-linear manifolds (e.g. Swiss-roll)	Uses kernel trick (RBF, sigmoid, etc.) to capture curves

Quick Reference Formulae

$$\mathbf{X}_c = \mathbf{X} - \mathbf{1}\boldsymbol{\mu}^\top, \quad \mathbf{X}_c = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top, \quad \mathbf{Z} = \mathbf{X}_c\mathbf{W}_d, \quad \hat{\mathbf{X}} = \mathbf{Z}\mathbf{W}_d^\top + \boldsymbol{\mu}$$

Take-away. PCA always boils down to eigen-decomposing the covariance matrix (or, numerically, applying SVD to $\mathbf{X}_c$). Centre $\rightarrow$ decompose $\rightarrow$ choose $d \rightarrow$ project.

10.3 Other Techniques

- **Locally Linear Embedding (LLE)** is a manifold learning technique that does not rely on projections like the previous algorithms.
- **Multidimensional Scaling (MDS)** reduces dimensionality while trying to preserve the distances between instances.
- **Isomap** creates a graph by connecting each instance to its nearest neighbours, then reduces dimensionality while trying to preserve the *geodesic distances* between instances.
- ***t*-Distributed Stochastic Neighbour Embedding (t-SNE)** reduces dimensionality while trying to keep similar instances close and dissimilar instances apart. Mostly used for visualisation.

11 Clustering

- **Goal**: identify similar instances and assign them to clusters. Similar to classification but clustering is an unsupervised task, with no know clusters.
- There is no universal definition of what a cluster is: it really depends on the context, and different algorithms will capture different kinds of clusters (K-Means uses centroids, DB scan uses continuous regions of densely packed instances.

11.1 K-Means Clustering

- **Goal**: Clusters instances around a central point, the *centroid*. We can then give give clusters a label (semi-supervised).
- **Convergence**: Guaranteed to converge in a finite number of steps since the mean squared distance between the instances and their closest centroids can only go down at each step.
- **Metrics**:
  - *inertia*: sum of squared distances between instances and their closest centroid. Want to minimise.
  - *score*: Negative inertia. Want to maximise.
- **Algorithm Steps:**
  1. Define (manually) a suitable number (k) of clusters.
  2. Start by placing these k centroids randomly (e.g, by randomly picking k instances from the dataset and use them as the centroids).
  3. Assign each data instance to the closest cluster centroid (usually Euclidean).
  4. Update each cluster centroid based on the data instances that have been assigned to it.
  5. Go to Step 3 and repeat until the cluster centroids stop moving.
- **Visualisation**: Done through a *Voronoi diagram* which shows decision boundaries. Decision boundary is perpendicular to a line going from c_1 to c_2 (equidistant).
- **Hard VS Soft Clustering**:
  - **Hard clustering**: Each instance is assigned to a single cluster. So, if there are m instances and k clusters, we get a vector of m elements, each of which has a cluster ID value.
  - **Soft Clustering**: Each instance receives a score for each cluster (can be distance or inverse of distance). For m instances and k clusters, we need an $m \times k$ matrix to store the scores.

Drawback 1 Sensitive to the initialisation of cluster centroids. While it always converges, it may not converge to global optimum. Overcome by:

1. Provide initial locations for the clusters if you know approximate cluster locations. `good_init`.
2. Run algorithm multiple times with different random initialisations and keep the best solution, either has lowest *inertia* or highest *score*. `n_init`.
3. Use the ***k-means++*** algorithm (David Arthur and Sergei Vassilvitskii, 2007), which stochastically selects centroids that are distant from one another. Scikit-Learn's `KMeans` uses this method by default.

Drawback 2 An optimal number of clusters k needs to be known and *Inertia* is not a good measure in choosing k because more k overfits (better for comparing with same k). Overcome by using *silhouette score*.

- Use the ***silhouette score*** which is the mean *silhouette coefficient* (higher is better) over all instances.
- The *silhouette coefficient* of an instance x_i is defined as:

$$sc_i = \frac{(b-a)}{\max(a,b)}$$

  Where:

  * a is the mean distance of x_i to other instances in the same cluster.
  * b is the mean distance of x_i to the instances of the next closest cluster.

- *silhouette coefficient* can be between -1 and 1:
  * Close to +1: instance is well inside its own cluster and far from other clusters.
  * Close to 0: the instance may be close to a cluster boundary
  * Close to -1: the instance may have been assigned to the wrong cluster.
- *silhouette Diagram* shows knives per cluster. A good model will have knives equal in thickness and tips above the mean silhouette score.

Drawback 3 k-means does not behave very well when the clusters have varying sizes, different densities, or nonspherical shapes. Over come by:

- ***Gaussian mixture model***, where elliptical clusters can be handled, as the model also estimates the covariance matrix of each cluster.

11.2 DB Scans

- **Goal**: Density-based spatial clustering of applications with noise (DBSCAN) defines clusters as continuous regions of high density of any shape.
- **Hyperparameters**: `epsilon` ϵ and `min_samples`.
- **Algorithm Steps:**
  1. For each instance, count how many other instances fall within a distance ϵ from it. This region is called the instance's ϵ-neighbourhood.
  2. If an instances ϵ-neighbourhood is large than `min_samples`, it is considered a *core instance*.
  3. Instances with core instances in its ϵ-neighbourhood form a cluster.
  4. Any instance with no core instance in its neighbourhood are anomalies.

Drawback 1 If there is no sufficiently low-density region around clusters or if denisity varies significantly across the clusters, DBSCAN can struggle to capture clusters properly.

Drawback 2 Computational complexity is $O(m^2 n)$, meaning it does not scale well with large datasets m^2.